Concept 5: Batch Processing

A **REST endpoint** responds when someone asks. The front-end says "give me this RFQ" and the service responds immediately. It's reactive — someone knocks, you answer.

A **batch process** is the opposite. Nobody asks. It runs **on a schedule**, on its own, automatically. Like an alarm clock.

For your reminders, it works like this:

```
Every day at 8:00 AM (or whatever schedule):
  1. Batch wakes up
  2. Lists all active reminders
  3. For each reminder, checks: is this RFQ overdue?
  4. If yes → trigger a notification (email, alert, etc.)
  5. Goes back to sleep
```

No front-end involved. No HTTP request. No user clicking anything. The batch just runs, does its job, and stops.

Imagine you write a function:

```
function getOverdueRfqs() {
    // looks in the database
    // finds all RFQs past their deadline
    // returns them
}
```

That function is your **domain logic**. It doesn't care who calls it. It just does its job and returns results.

Now, **who can call this function?**

Scenario A: The front-end user clicks a button "Show overdue RFQs." That HTTP request hits your REST adapter, and your REST code calls `getOverdueRfqs()` and sends the result back to the browser.

Scenario B: It's 8:00 AM. Nobody clicked anything. The batch scheduler wakes up automatically, calls the exact same `getOverdueRfqs()`, and for each result sends an email notification.

Same function. Two different callers. That's it.

Now the "two mains" part. When you start your application, something has to launch. A "main" is just the starting point:

```
Main 1 → starts the REST server (sits and waits for front-end calls)
Main 2 → starts the batch scheduler (sits and waits for the clock)
```

Both live in the same codebase. Both can call `getOverdueRfqs()` because it's right there in the same project. No REST call needed between them. No second database. No duplication.

If reminders were a **separate microservice**, Main 2 would need to call Main 1 over HTTP to get RFQ data — that's the overhead your bosses want to avoid.

Without hexagonal architecture, you might write your REST code like this:

```
// REST endpoint code
app.get('/rfqs/overdue', (req, res) => {
    const rfqs = database.query("SELECT * FROM rfq WHERE deadline < NOW()");
    res.send(rfqs);
});
```

The business logic (finding overdue RFQs) is **mixed into** the REST code. Now when you want the batch to do the same thing, what do you do? You copy-paste:

```
// Batch code
schedule('8am', () => {
    const rfqs = database.query("SELECT * FROM rfq WHERE deadline < NOW()");
    sendEmails(rfqs);
});
```

Same query, duplicated in two places. Tomorrow the business rule changes — "overdue means 3 days past deadline, not 1." You have to update it in **both** places. That's the double maintenance your bosses warned about.

Hexagonal architecture says: **pull that logic out into the center.**

```
// DOMAIN (center) — business logic lives here alone
function getOverdueRfqs() {
    return database.query("SELECT * FROM rfq WHERE deadline < NOW()");
}

// ADAPTER 1: REST — just calls the domain
app.get('/rfqs/overdue', (req, res) => {
    res.send(getOverdueRfqs());
});

// ADAPTER 2: Batch — just calls the domain
schedule('8am', () => {
```

```
    sendEmails(getOverdueRfqs());
});
```

The business rule exists in **one place only**. The adapters are thin — they just receive the request (from HTTP or from the clock) and pass it to the domain. If the rule changes, you change it once.

That's hexagonal architecture: **the domain doesn't know or care who's calling it.** The adapters are just different doors into the same room.

but the question when i have this reflex of choosing this type of approach (adapter)?